

Hands-On: Unit Testing and Component Testing in React

Approach 1: Unit Testing with Jest

Jest is a popular testing framework used for unit testing in React. To get started with Jest, you'll need to install it using npm or yarn:

```
npm install --save-dev jest
```

Once Jest is installed, you can create a test file with the `.test.js` extension and write your tests. Here's an example of a test file for a simple React component:

```
import React from 'react';
import { render, screen } from '@testing-library/react';
import App from './App';

describe('App', () => {
  test('renders the app header', () => {
    render(<App />);
    const headerElement = screen.getByText(/Welcome to My App/i);
    expect(headerElement).toBeInTheDocument();
  });
});
```

In this example, we use the `render` function from `@testing-library/react` to render our `App` component and then use the `getByText` function to find the app header element. We then use the `toBeInTheDocument` matcher from Jest to check if the header element is present in the DOM.

Approach 2: Component Testing with React Testing Library

React Testing Library is another popular testing library used for component testing in React. To get started with React Testing Library, you'll need to install it using npm or yarn:

```
npm install --save-dev @testing-library/react
```

Once React Testing Library is installed, you can create a test file with the `.test.js` extension and write your tests. Here's an example of a test file for a simple React component:

```
import React from 'react';
import { render, screen } from '@testing-library/react';
import Button from './Button';

describe('Button', () => {
  test('calls the onClick handler when clicked', () => {
    const onClickMock = jest.fn();
    render(<Button onClick={onClickMock}>Click Me</Button>);
    const buttonElement = screen.getByRole('button', { name: /Click Me/i });
    buttonElement.click();
    expect(onClickMock).toHaveBeenCalled();
  });
});
```

In this example, we create a mock function using the `jest.fn()` method and pass it as a prop to our `Button` component. We use the `getByRole` function to find the button element and then simulate a click on it using the `click` method. Finally, we use the `toHaveBeenCalled` matcher to check if the `onClick` mock function was called.

Conclusion

In this lab, you learned about different approaches to testing in React, including unit testing with Jest and component testing with React Testing Library. With these tools, you can write robust and reliable tests for your React components and ensure the quality of your code.